CRYPTA : Examen Réparti 1

Durée : 2h

25 novembre 2021

- Notes manuscrites et planches des cours ISEC et CRYPTA autorisées à l'exclusion de toute autre document.
- L'utilisation de tout matériel électronique (en dehors d'une montre non connectée) est interdite.
- Il y a trois exercices indépendants.
- Une rédaction claire et concise sera appréciée. Toute affirmation devra être justifiée.
- Une question non résolue n'empêche pas de faire les suivantes. (dans ce cas indiquez clairement que vous admettez le(s) résultat(s) de la question non faite).
- Le sujet est certainement trop long. Une estimation au doigt mouillé suggère que seuls 0.000000013% des êtres humains peuvent finir le sujet en 2h. Traitez les exercices que vous préférez d'abord.

1 Fonction de hachage et preuve à divulgation nulle de connaissance

Soit $\mathbb{G}$ un groupe fini d'ordre premier q et soient g et h deux éléments de $\mathbb{G}$. Nous considérons la fonction suivante :

$$\begin{array}{rcl} \mathcal{H} \ : \mathbb{Z}/q\mathbb{Z}^2 & \longrightarrow & \mathbb{G} \\ (x,y) & \longmapsto & g^x h^y \end{array}$$

▷ **Question 1:** Montrer que si il existe un algorithme efficace capable de trouver une collision pour la fonction $\mathcal{H}$ (c'est-à-dire deux couples (x_1, y_1) et (x_2, y_2) de $\mathbb{Z}/q\mathbb{Z}^2$ tels que $\mathcal{H}(x_1, y_1) = \mathcal{H}(x_2, y_2)$ avec $(x_1, y_1) \neq (x_2, y_2)$), alors il existe un algorithme efficace capable de trouver le logarithme discret de h en base g.

Étant donné un élément γ de $\mathbb{G}$, une pré-image de γ pour la fonction de hachage $\mathcal{H}$ est un couple (x, y) de $\mathbb{Z}/q\mathbb{Z}^2$ tel que $\mathcal{H}(x, y) = g^x h^y = \gamma$. Supposons qu'un utilisateur veut prouver qu'il connaît une pré-image pour la fonction de hachage $\mathcal{H}$ mais sans révéler d'information sur cette pré-image. Nous allons étudier trois protocoles qui sont des variantes du protocole de Schnorr.

Considérons tout d'abord le protocole suivant :

Phase d'engagement : Le prouveur tire uniformément aléatoirement $k, \ell \in \mathbb{Z}/q\mathbb{Z}$ et calcule $r = g^k \in \mathbb{G}$ et $s = h^\ell \in \mathbb{G}$. Il envoie le couple (r, s) au vérifieur.

Phase de challenge : Le vérifieur tire uniformément aléatoirement $c \in \mathbb{Z}/q\mathbb{Z}$ et envoie c au prouveur.

Phase de réponse : Le prouveur calcule $u = k - xc \bmod q$ et $v = \ell - yc \bmod q$. Il envoie (u, v) au vérifieur. Celui accepte le prouveur si et seulement si $\gamma^c g^u h^v = rs$.

▷ **Question 2:** Montrer que ce protocole est complet et significatif.

▷ **Question 3:** Montrer que ce protocole divulgue de l'information (par exemple que le vérifieur peut apprendre la valeur g^x ou h^y).

Considérons désormais le protocole suivant :

Phase d'engagement : Le prouveur tire uniformément aléatoirement $k \in \mathbb{Z}/q\mathbb{Z}$ et calcule $r = g^k h^k = (gh)^k \in \mathbb{G}$. Il envoie l'élément r au vérifieur.

Phase de challenge : Le vérifieur tire uniformément aléatoirement $c \in \mathbb{Z}/q\mathbb{Z}$ en envoie c au prouveur.

Phase de réponse : Le prouveur calcule $u = k - xc \bmod q$ et $v = k - yc \bmod q$. Il envoie (u, v) au vérifieur. Celui accepte le prouveur si et seulement si $\gamma^c g^u h^v = r$.

▷ **Question 4:** Montrer que ce protocole est complet et significatif.

▷ **Question 5:** Montrer que ce protocole divulgue de l'information (par exemple que le vérifieur peut apprendre la valeur $(x - y) \bmod q$).

Considérons enfin le protocole suivant :

Phase d'engagement : Le prouveur tire uniformément aléatoirement $k, \ell \in \mathbb{Z}/q\mathbb{Z}$ et calcule $r = g^k h^\ell \in \mathbb{G}$. Il envoie l'élément r au vérifieur.

Phase de challenge : Le vérifieur tire uniformément aléatoirement $c \in \mathbb{Z}/q\mathbb{Z}$ en envoie c au prouveur.

Phase de réponse : Le prouveur calcule $u = k - xc \bmod q$ et $v = \ell - yc \bmod q$. Il envoie (u, v) au vérifieur. Celui accepte le prouveur si et seulement si $\gamma^c g^u h^v = r$.

▷ **Question 6:** Montrer que ce protocole est complet et significatif.

▷ **Question 7:** Montrer que ce protocole est à divulgation nulle de connaissance.

2 Signature RSA avec bourrage constant

On considère un schéma de signature basé sur RSA. La clef publique N fait n bits de long, et on considère un autre entier M, sensiblement plus petit que N. Les messages qu'on peut signer sont les entiers compris entre zéro et $M - 1$. Il y a aussi une constante P, qui est un nombre fixé, plus grand que M. La signature d'un message m est $s := (P + m)^d \bmod N$.

Pour fixer les idées, on peut imaginer N de 2048 bits, $M = 2^{1536}$, $P =$ `0xDeadBeef0000FFFF0000.....0000FFFF` (2048 bits de long).

▷ **Question 8:** Comment vérifie-t-on que la signature est correcte ?

▷ **Question 9:** Rappelez comment fonctionne l'attaque triviale par malléabilité sur la signature RSA naïve ("textbook").

▷ **Question 10:** Expliquer pourquoi l'ajout du bourrage P empêche cette attaque directe.

On va maintenant voir une attaque qui permet de contrefaire des signatures malgré la présence du bourrage. On va fabriquer trois messages m_1, m_2 et m_3 ; on en fera signer deux par un oracle de signature ; enfin on produira nous-même la signature du troisième message. Ceci casse le schéma de signature

▷ **Question 11:** Quelle notion précise de sécurité est-elle brisée ?

Notons s_i la signature du message m_i. Clairement, on a $s_1 \equiv s_2 s_3 \bmod N$ si et seulement si

$$(P + m_1) \equiv (P + m_2)(P + m_3) \bmod N. \qquad (\clubsuit)$$

L'attaque fonctionne en produisant m_1, m_2 et m_3 qui satisfont cette relation et qui sont dans l'intervalle $[0, M[$.

▷ **Question 12:** On tire m_1 et m_2 au hasard dans $\{0, 1, \ldots, M - 1\}$. Quelle valeur de m_3 satisfait l'équation $(\clubsuit)$? Pourquoi cela ne permet-il pas *a priori* de casser le schéma de signature ?

On procède de la façon suivante : on choisit $m_3 = 0$ puis on va calculer, par un moyen qui reste à déterminer, m_1 et m_2 qui satisfont la relation ci-dessus et les contraintes de taille.

▷ **Question 13:** Justifier que deux entiers m_1, m_2 satisfont l'équation $(\clubsuit)$ si et seulement s'il existe un entier k tel que :

$$(P - 1) + m_2 = m_1 P^{-1} + kN \qquad (\heartsuit)$$

On considère le réseau euclidien engendré par les lignes de :

$$A = \begin{pmatrix} 1 & P^{-1} \\ & N \end{pmatrix}$$

▷ **Question 14:** Justifier que si m_1, m_2 et k satisfont $(\heartsuit)$, alors $(m_1, k)A = (m_1, P - 1 + m_2)$.

▷ **Question 15:** Quel est le volume du réseau ? Quelle est (à peu près) la norme de son plus court vecteur ?

▷ **Question 16:** Quelle est la distance entre $(m_1, k)A$ et $(0, P-1)$?

▷ **Question 17:** Résoudre le problème CVP dans ce réseau est-il difficile ?

▷ **Question 18:** Considérons le point du réseau le plus proche de $(0, P-1)$. Quelles sont les tailles de m_1 et m_2 correspondantes ?

▷ **Question 19:** Donner un algorithme qui permet de forger le schéma de signature avec bourrage.

3 Un PRNG (peut-être) ultra-sûr

Vous êtes expert-évaluateur dans une agence étatique. L'entreprise `Sn4k30il S3cur1ty` a développé un générateur pseudo-aléatoire, et vous devez écrire un rapport sur son niveau de sécurité.

Il possède un état interne de 8×64 bits (représenté par les variables a, b, c, d, e, f, g, h), et il produit des sorties de 64 bits (z) à chaque itération. Sa graine est la valeur initiale de l'état interne. Voici son pseudo-code

```
def prng(a, b, c, d, e, f, g, h):
    prime = 2**64 - 59
    while True:
        ######### Produit matrice-vecteur
        i =   7886*a  +7612*b +26661*c -15282*d +32867*e +48473*f -36291*g +35350*h
        j = -57569*a +19465*b -24992*c  -1812*d +26618*e +29956*f +34577*g -18859*h
        k =  -2446*a +61735*b -22427*c +34186*d  +6992*e  -9478*f  +8197*g -27775*h
        l =  -1185*a  -6312*b -56926*c +15565*d +11059*e  +1553*f -57153*g  +5707*h
        m =   -535*a  -1791*b -39310*c +43305*d  +6273*e +56313*f  -3904*g +17665*h
        n = -10209*a +22995*b +20231*c  +1049*d -14251*e -41664*f +12854*g +33455*h
        o =  23582*a -29517*b -16622*c +16105*d +39186*e +13879*f +16248*g +11659*h
        p =  11671*a  +5093*b -13453*c -33945*d  +6320*e +22666*f +28441*g +25619*h
        ######### Arrondi ("learning with rounding")
        q = round(i / prime)
        r = round(j / prime)
        s = round(k / prime)
        t = round(l / prime)
        u = round(m / prime)
        v = round(n / prime)
        w = round(o / prime)
        x = round(p / prime)
        ######### Extraction de la sortie (produit scalaire)
        y = 0
        y = y -  100539497500637377139787102742151 * q
        y = y - 438519913312755989101196347924980 8 * r
        y = y + 271430193339396034182303411902832 4 * s
        y = y -  893200328965818184420614031056548 * t
        y = y -  392009643508364887148804587723243 * u
        y = y - 242104782892814722528387825226478 8 * v
        y = y + 156612503311341570504596335797497 6 * w
        y = y + 277147370977466605324739314333243 5 * x
        y = y * 788355926932909359757738600762453384 65
        z = (y >> 64) & 0xffffffffffffffff
        ######### mise à jour de l'état interne
        a = b
        b = c
        c = d
        d = e
        e = f
        f = g
        g = h
        h = z
        yield z                    # si vous ne savez pas ce qu'est "yield", imaginez que c'est "print"
```

Ses concepteurs affirment qu'il est sûr car il reposerait sur des réseaux euclidiens, des nombres premiers, de l'arithmétique modulaire, la difficulté de la factorisation, du Learning With Rounding post-quantique, etc.

Mais *vous*, vous savez qu'il est extraordinairement faible. Il vous suffit d'expliquer... pourquoi.